

字符串

字符串哈希
前缀函数与kmp
Manacher
Trie字典树
Z函数
AC自动机
后缀数组
后缀自动机
回文自动机

字符串

字符串哈希

字符串中一般 $base = 131$, $mod = \infty$ (均要求为素数)

多项式Hash函数分 $f(s) = \sum_{i=1}^l s[i] \times b^{l-i} \pmod{M}$

多次询问子串哈希: 可预处理出一个 b 进制数 $\pmod{M}$ 的数组, $f(s[l \dots r]) = f(r) - f(l-1) \times b^{r-l+1}$, 复杂度 $O(1)$

降低哈希碰撞可用双hash

(k 进制异或哈希)

```
1  array<int, len> base = {131, 233}, mod = {1'000'000'007, 998'244'353};
2
3  template <int len = 2>
4  struct Hash { //双哈希
5      vector<array<int, len>> pre, pw;
6      Hash(string s = " ") { // 传入的 s 必须重 1 开始
7          pw = pre = vector(s.size(), array<int, len>());
8          fill(pw[0].begin(), pw[0].end(), 1);
9          for (int i = 1; i < s.size(); i++) {
10             for (int _ = 0; _ < len; _++) {
11                 (pre[i][_] = pre[i-1][_]*base[_]+s[i]) %= mod[_];
12                 (pw[i][_] = pw[i-1][_]*base[_]) %= mod[_];
13             }
14         }
15     }
16
17     array<int, len> query(int l, int r, array<int, len> res = {0}) {
18         for (int _ = 0; _ < len; _++) {
19             (((res[_] = pre[r][_]-pre[l-1][_]*pw[r-l+1][_]) %= mod[_]) += mod[_]) %= mod[_];
20         }
21         return res;
22     }
23 };
```

随机化哈希: 速度相比双哈希快很多, 错误概率小, 将普通哈希的 $base$ 换成一个随机值, 采用 `unsigned long long` 来自然溢出不出现负数

```
1  mt19937 rd(time(0));
2  vector<unsigned long long> base;
3
4  [&](int mx) {
5      base = vector<unsigned long long>(mx+1);
6      for (auto &x : base) {
7          x = rd();
8      }
9  } ();
```

```

10
11 auto Hash = [&](auto s) { //传入的 s 数组必须下标从 1 开始
12     vector<unsigned long long> pre(s.size());
13     for (int i = 1; i < s.size(); i++) {
14         pre[i] = pre[i - 1] + s[i] * base[i];
15     }
16     return Hash;
17 };
18
19 auto query = [&](int l, int r, vector<unsigned long long> pre) {
20     return pre[r] - pre[l - 1];
21 };

```

前缀函数与kmp

```

1 struct KMP : vector<signed> { // 数组表示 S[:i] 最长真前缀和真后缀的长度
2     KMP(const string & s = {}) : vector(s.size()) { // 从 0 开始, 前缀函数
3         for (int i = 1; i < size(); at(i++) += s[i] == s[at(i)]) {
4             for (at(i) = at(i - 1); at(i) && s[i] != s[at(i)]; at(i) = at(at(i) - 1));
5         }
6     }
7     KMP(const string & t, const string & s, char o = '#') : KMP(t + o + s) {}
8 };
9
10 struct KMP_ZFun : vector<signed> {
11     vector<signed> z, uni; // 跳到下个 kmp[i] != kmp[kmp[i] - 1] 不同字母的位置
12     void Dy_ZFun(const string & s, int i) { // i 从 [0, n - 1] 都要
13         emplace_back(0), z.emplace_back(-1), uni.emplace_back(0); // z[i] == -1 ? s.size() - i :
14         z[i];
15         if (i) {
16             for (at(i) = at(i - 1); at(i) && s[i] != s[at(i)]; at(i) = at(at(i) - 1));
17             at(i) += s[i] == s[at(i)];
18             if (at(i) >= 1) uni[i] = s[at(i)] != s[at(at(i) - 1)] ? at(at(i) - 1) : uni[at(i) - 1];
19             for (int k = at(i - 1); k; k = (s[i] == s[at(k - 1)] ? uni[k - 1] : at(k - 1))) {
20                 if (s[i] != s[k]) z[i - k] = k;
21             }
22             if (s[i] != s[0]) z[i] = 0;
23         }
24     };
25
26 struct ZFun : vector<signed> { // 数组表示 S 与 S[:i] 最长公共前缀 (LCP)
27     ZFun(const string & s = {}) : vector(s.size()) { // 从 0 开始
28         at(0) = s.size();
29         for (int i = 1, j = 1; i < size(); i++) {
30             for (at(i) = max<int>(0, min<int>(at(j) + j - i, at(i - j))); i + at(i) < size() &&
31             s[at(i)] == s[i + at(i)]; at(i)++);
32             if (i + at(i) > j + at(j)) j = i;
33         }
34         ZFun(const string & t, const string & s, char o = '#') : ZFun(t + o + s) {}
35     };
36
37 struct Manacher : vector<signed> { // 从 0 开始, 得到所在的回文串总长度 (去除 o)
38     Manacher(const string & s = {}, char o = '#') : vector(2 * s.size() + 1) {
39         string t(1, o);
40         for (auto & c : s) t += c, t += o;
41         for (int i = 0, j = 0; i < size(); i++) {
42             if (2 * j - i >= 0 && j + at(j) > i) at(i) = min(at(2 * j - i), at(j) + j - i);
43             for (; i - at(i) >= 0 && i + at(i) < size() && t[i - at(i)] == t[i + at(i)]; at(i)++);
44             if (i + at(i) > j + at(j)) j = i;
45         }
46         for (auto & x : *this) x--;
47     }
48 };

```

前缀函数：

前缀函数 $\pi[i]$ 是子串 $s[0 \dots i]$ 最长的真前缀和真后缀的长度, $\pi[0] = 0$

计算前缀函数： 时间复杂度 $O(n)$

应用： kmp

作用： 给定一个文本 t 和一个字符串 s , 找出 s 在 t 中的所有出现, 记 t 长度 n , s 长度 m

构造 $s + \#$ 并计算其的前缀函数, $\#$ 的作用防止匹配长度超过 n

并在 $s + \#$ 后加入 t , 计算 $i > n$ 时的前缀函数, 为 n 时则成功匹配, 在 t 中位置为

$i - (n - 1) - (n + 1) = i - 2 * n$, 此时 $i - 2 * n$ 为字符串从0开始计数的, 从1开始需要+1

故可一次性读入 t 并计算当前位置的前缀函数值, 内存 n

时间复杂度: $O(n + m)$, 空间复杂度 $O(n)$

性质： 假设 s 的长度为 $len \Rightarrow s$ 存在最小循环节, 循环节长度为 $cyclelen = len - next[len]$, 循环子串为 $s[0 \dots len - next[len]]$

(i) 若 $len - next[len] \mid len \Rightarrow str$ 可以由循环节循环组成

(ii)

若 $len - next[len] \nmid len \Rightarrow$ 还需要添加 $cyclelen - len \% cyclelen$ 个字母, 需要补充的字符串为 $s[len \% cyclelen \dots cyclelen - len \% cyclelen]$

统计每个前缀的出现次数：

在该节我们将同时讨论两个问题。给定一个长度为 n 的字符串 s , 在问题的第一个变种中我们希望统计每个前缀 $s[0 \dots i]$ 在同一个字符串的出现次数, 在问题的第二个变种中我们希望统计每个前缀 $s[0 \dots i]$ 在另一个给定字符串 t 中的出现次数。

首先让我们来解决第一个问题。考虑位置 i 的前缀函数值 $\pi[i]$ 。根据定义, 其意味着字符串 s 一个长度为 $\pi[i]$ 的前缀在位置 i 出现并以 i 为右端点, 同时不存在一个更长的前缀满足前述定义。与此同时, 更短的前缀可能以该位置为右端点。容易看出, 我们遇到了在计算前缀函数时已经回答过的问题: 给定一个长度为 j 的前缀, 同时其也是一个右端点位于 i 的后缀, 下一个更小的前缀长度 $k < j$ 是多少? 该长度的前缀需同时也是一个右端点为 i 的后缀。因此以位置 i 为右端点, 有长度为 $\pi[i]$ 的前缀, 有长度为 $\pi[\pi[i] - 1]$ 的前缀, 有长度为 $\pi[\pi[\pi[i] - 1] - 1]$ 的前缀, 等等, 直到长度变为0。故而我们可以通过下述方式计算答案。

```
1 vector<int> ans(n + 1);
2 for (int i = 0; i < n; i++) ans[pi[i]]++;
3 for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];
4 for (int i = 0; i <= n; i++) ans[i]++;
```

在上述代码中我们首先统计每个前缀函数值在数组 π 中出现了多少次, 然后再计算最后答案: 如果我们知道长度为 i 的前缀出现了恰好 $ans[i]$ 次, 那么该值必须被叠加至其最长的既是后缀也是前缀的子串的出现次数中。在最后, 为了统计原始的前缀, 我们对每个结果加1。

现在考虑第二个问题。我们应用来自 Knuth-Morris-Pratt 的技巧: 构造一个字符串 $s + \# + t$ 并计算其前缀函数。与第一个问题唯一的不同之处在于, 我们只关心与字符串 t 相关的前缀函数值, 即 $i \geq n + 1$ 的 $\pi[i]$ 。有了这些值之后, 我们可以同样应用在第一个问题中的算法来解决该问题。

一个字符串中本质不同子串的数目：

给定一个长度为 n 的字符串 s , 我们希望计算其本质不同子串的数目。

我们将迭代地解决该问题。换句话说, 在知道了当前的本质不同子串的数目的情况下, 我们要找出一种在 s 末尾添加一个字符后重新计算该数目的方法。

令 k 为当前 s 的本质不同子串数量。我们添加一个新的字符 c 至 s 。显然, 会有一些新的子串以字符 c 结尾。我们希望对这些以该字符结尾且我们之前未曾遇到的子串计数。

构造字符串 $t = s + c$ 并将其反转得到字符串 $t^{\sim}$ 。现在我们的任务变为计算有多少 $t^{\sim}$ 的前缀未在 $t^{\sim}$ 的其余任何地方出现。如果我们计算了 $t^{\sim}$ 的前缀函数最大值 $\pi_{\max}$, 那么最长的出现在 s 中的前缀其长度为 $\pi_{\max}$ 。自然的, 所有更短的前缀也出现了。

因此, 当添加了一个新字符后新出现的子串数目为 $|s| + 1 - \pi_{\max}$ 。

所以对于每个添加的字符, 我们可以在 $O(n)$ 的时间内计算新子串的数目, 故最终复杂度为 $O(n^2)$ 。

值得注意的是, 我们也可以重新计算在头部添加一个字符, 或者从尾或者头移除一个字符时的本质不同子串数目。

字符串压缩：

给定一个长度为 n 的字符串 s , 我们希望找到其最短的「压缩」表示, 也即我们希望寻找一个最短的字符串 t , 使得 s 可以被 t 的一份或多份拷贝的拼接表示。

显然，我们只需要找到 t 的长度即可。知道了该长度，该问题的答案即为长度为该值的 s 的前缀。

让我们计算 s 的前缀函数。通过使用该函数的最后一个值 $\pi[n-1]$ ，我们定义值 $k = n - \pi[n-1]$ 。我们将证明，如果 k 整除 n ，那么 k 就是答案，否则不存在一个有效的压缩，故答案为 n 。

假定 n 可被 k 整除。那么字符串可被划分为长度为 k 的若干块。根据前缀函数的定义，该字符串长度为 $n - k$ 的前缀等于其后缀。但是这意味着最后一个块同倒数第二个块相等，并且倒数第二个块同倒数第三个块相等，等等。作为其结果，所有块都是相等的，因此我们可以将字符串 s 压缩至长度 k 。

KMP 自动机：

KMP 自动机就是一个不断读入待匹配串，每次匹配时走到接受状态的 DFA。如果共有 m 个状态，第 i 个状态表示已经匹配了前 i 个字符。我们定义 $\text{trans}_{i,c}$ 表示状态 i 读入字符 c 后到达的状态， next_i 表示 prefix function，则有：

$$\text{trans}_{i,c} = \begin{cases} i + 1, & \text{if } b_i = c \\ \text{trans}_{\text{next}_i, c}, & \text{otherwise} \end{cases}$$

(约定 $\text{next}_0 = 0$)

我们发现 trans_i 只依赖于之前的值，所以可以跟 KMP 一起求出来。

需要注意走到接受状态之后应该立即转移到该状态的 next 。

时间和空间复杂度： $O(m|\Sigma|)$ 。

Manacher

作用

对于一个长度为 n 的字符串（下标从 1 开始，可以找出 d_i 分别表示奇回文子串以位置 i 开始到最右端位置包含的字符个数，即以 i 位置为中心的最长奇回文子串半径（半径包含自己）

对于偶回文子串，我们通过对每个字符前后加上 #（任意一个原字符串中未出现过的字符）来转变为奇回文子串，这样我们只需要考虑奇回文子串足以

此处 $d_i - 1$ 意义为以 i 位置为中心在**原串**中的最长回文子串长度， $\lfloor \frac{d_i}{2} \rfloor$ 为以 i 位置为中心在**原串**中的回文子串个数

空间复杂度： $O(n)$

时间复杂度： $O(n)$

算法原理

假设 $d_1 \sim d_{i-1}$ 已经计算完毕，维护以 $1 \sim i-1$ 为中心的最靠右的回文子串的边界 $[l, r]$ 和区间中点 mid ，实际上 $l = 2 \times mid - r$ ，故不记录 l

- 若 $i > r$ ，我们暴力计算回文子串的半径，即一步步比较
- 若 $i \leq r$ ，我们考虑从 $[l, r]$ 中获取信息
 - 若 i 在 $[l, r]$ 中能**找到反转位置**，得到反转位置 $j = l + (r - i) = 2 \times mid - i$ ，故 $d_i = d_j$

$$\dots \overbrace{s_l \dots s_{j-d_1[j]+1} \dots s_j \dots s_{j+d_1[j]-1} \dots s_{i-d_1[j]+1} \dots s_i \dots s_{i+d_1[j]-1} \dots s_r}^{\text{palindrome}} \dots$$

$$\underbrace{s_l \dots s_{j-d_1[j]+1} \dots s_j \dots s_{j+d_1[j]-1}}_{\text{palindrome}} \quad \underbrace{s_{i-d_1[j]+1} \dots s_i \dots s_{i+d_1[j]-1}}_{\text{palindrome}} \dots s_r$$

- 若 i 在 $[l, r]$ 中**无法找到反转位置**，这种情况我们进行截断操作，先令 $d_i = r - i$ ，因为此长度是最长不会超过 $[l, r]$ 的回文子串半径，然后再暴力一步步的增加回文子半径

$$\dots \overbrace{s_l \dots s_j \dots s_{j+(j-l)} \dots s_{i-(r-i)} \dots s_i \dots s_r}^{\text{palindrome}} \dots$$

$$\underbrace{s_l \dots s_j \dots s_{j+(j-l)}}_{\text{palindrome}} \quad \underbrace{s_{i-(r-i)} \dots s_i \dots s_r}_{\text{palindrome}} \quad \underbrace{\dots}_{\text{try moving here}}$$

- 最后我们更新新的 $[l, r], mid$

注意到时间复杂度瓶颈主要在于暴力比较并计算回文子串半径这里，由于每次暴力比较均会使得 r 增加，而 r 在算法过程不减小， $r \leq 2 \times n$ ，故时间复杂度为 $O(n)$ 利用回文子串有中心，时间复杂度 $O(n)$

Trie字典树

trie：可用求字符串检索，异或极值，异或和等问题

```

1 namespace TRIE
2 {
3     class Trie
4     {
5     public:
6         class LazyTag
7         {
8         public:
9             int o,a,x;
10             inline LazyTag():o(0),a(0),x(0)
11             {
12
13             }
14             inline LazyTag(const int o,const int a,const int x):o(o),a(a),x(x)
15             {
16
17             }
18             inline void Merge(const LazyTag &m)
19             {
20                 o^=o&m.a;
21                 a^=a&m.o;
22                 int xx=(o&m.x)|(a&m.x);
23                 o^=xx;
24                 o|=m.o;
25                 a^=xx;
26                 a|=m.a;
27                 x^=((x&m.a)|(x&m.o))^m.x^xx;
28             }
29             inline void Clean()
30             {
31                 o=a=x=0;
32             }
33         };
34         int son[2];
35         LazyTag tag;
36         inline void Clean()
37         {
38             tag.Clean();
39             son[0]=son[1]=0;
40         }
41     }trie[MAXN*32];
42     #define SON(x) trie[now].son[x]
43     int trie_cnt=0;
44     int root;
45     inline void Clean()
46     {
47         root=1;
48         trie_cnt=1;
49         trie[1].Clean();
50     }
51     inline void PushDown(const int now,const int dep);
52     void Merge(int &tree1,int &tree2,const int dep)
53     {
54         if(!tree1||!tree2)
55         {
56             tree1+=tree2;
57             tree2=0;
58             return;
59         }
60         if(dep==1)
61         {
62             tree2=0;
63             return;
64         }
65         PushDown(tree1,dep);
66         PushDown(tree2,dep);
67         Merge(trie[tree1].son[0],trie[tree2].son[0],dep-1);

```

```

68     Merge(trie[tree1].son[1], trie[tree2].son[1], dep-1);
69     tree2=0;
70 }
71 inline void PushDown(const int now, const int dep)
72 {
73     if(trie[now].tag.o & (1<<dep))
74     {
75         Merge(SON(1), SON(0), dep-1);
76     }
77     if(trie[now].tag.a & (1<<dep))
78     {
79         Merge(SON(0), SON(1), dep-1);
80     }
81     if(trie[now].tag.x & (1<<dep))
82     {
83         Swap(SON(0), SON(1));
84     }
85     if(SON(0))
86     {
87         trie[SON(0)].tag.Merge(trie[now].tag);
88     }
89     if(SON(1))
90     {
91         trie[SON(1)].tag.Merge(trie[now].tag);
92     }
93     trie[now].tag.Clean();
94 }
95 void Insert(const int val, int &now, const int dep=30)
96 {
97     if(!now)
98     {
99         now=++trie_cnt;
100         trie[now].Clean();
101     }
102     if(dep==1)
103     {
104         return;
105     }
106     PushDown(now, dep);
107     if(val & (1<<dep))
108     {
109         Insert(val, SON(1), dep-1);
110     }
111     else
112     {
113         Insert(val, SON(0), dep-1);
114     }
115 }
116 inline void Or(const int a)
117 {
118     trie[root].tag.Merge(Trie::LazyTag(a, 0, 0));
119 }
120 inline void And(const int a)
121 {
122     trie[root].tag.Merge(Trie::LazyTag(0, MAX^a, 0));
123 }
124 inline void Xor(const int a)
125 {
126     trie[root].tag.Merge(Trie::LazyTag(0, 0, a));
127 }
128 int QueryMax(const int val, const int now, const int dep=30, const int result=0)
129 {
130     if(!now)
131     {
132         return -1;
133     }
134     if(dep==1)

```

```

135     {
136         return result;
137     }
138     PushDown(now, dep);
139     bool p=val&(1<<dep);
140     if(SON(p^1))
141     {
142         return QueryMax(val, SON(p^1), dep-1, result|(1<<dep));
143     }
144     else
145     {
146         return QueryMax(val, SON(p), dep-1, result);
147     }
148 }
149 }
150 inline void Work()
151 {
152     Read(n,m);
153     TRIE::Clean();
154     int a;
155     REP(i,1,n)
156     {
157         Read(a);
158         TRIE::Insert(a, TRIE::root);
159     }
160     int opt;
161     REP(i,1,m)
162     {
163         Read(opt,a);
164         if(opt==1)
165         {
166             TRIE::Insert(a, TRIE::root);
167         }
168         if(opt==2)
169         {
170             TRIE::Or(a);
171         }
172         if(opt==3)
173         {
174             TRIE::And(a);
175         }
176         if(opt==4)
177         {
178             TRIE::Xor(a);
179         }
180         if(opt==5)
181         {
182             WriteLn(TRIE::QueryMax(a, TRIE::root));
183         }
184     }
185 }

```

```

1  struct trie_node {
2      array<int, 26> son = {};    //son是看字符
3      int exist = 0;    //记录以该节点结尾的字符串个数
4  };
5
6  vector<trie_node> tr(1);
7
8  auto insert = [&](string& s) {
9      int p = 0;
10     for (auto x : s) {
11         int c = x - 'a';
12         if (!tr[p].son[c]) {
13             tr[p].son[c] = tr.size();    //若没有就添加节点
14             tr.push_back({});
15         }

```

```

16         p = tr[p].son[c];    //向下查
17     }
18     tr[p].exist++;    //记录此处为字符串
19 };
20
21 auto find = [&](string& s) {
22     int p = 0;
23     for (auto x : s) {
24         int c = x - 'a';
25         if (!tr[p].son[c]) {
26             return (int)0;
27         }
28         p = tr[p].son[c];
29     }
30     return tr[p].exist;
31 };

```

Z函数

定义： 对于一个长度为 n 的字符串 s ，定义函数 $z[i]$ 表示 s 和 $s[i, n-1]$ （即以 $s[i]$ 开头的后缀）的最长公共前缀（LCP）的长度，则 z 被称为 s 的 Z 函数。特别地， $z[0] = 0$ 。

匹配所有子串：

为了避免混淆，我们将 t 称作文本，将 p 称作模式。所给出的问题是：寻找在文本 t 中模式 p 的所有出现（occurrence）。

为了解决该问题，我们构造一个新的字符串 $s = p + \diamond + t$ ，也即将 p 和 t 连接在一起，但是在中间放置了一个分割字符 $\diamond$ （我们将如此选取 $\diamond$ 使得其必定不出现在 p 和 t 中）。

首先计算 s 的 Z 函数。接下来，对于在区间 $[0, |t| - 1]$ 中的任意 i ，我们考虑以 $t[i]$ 为开头的后缀在 s 中的 Z 函数值 $k = z[i + |p| + 1]$ 。如果 $k = |p|$ ，那么我们知道有一个 p 的出现位于 t 的第 i 个位置，否则没有 p 的出现位于 t 的第 i 个位置。

其时间复杂度（同时也是其空间复杂度）为 $O(|t| + |p|)$ 。

本质不同子串数：

给定一个长度为 n 的字符串 s ，计算 s 的本质不同子串的数目。

考虑计算增量，即在知道当前 s 的本质不同子串数的情况下，计算出在 s 末尾添加一个字符后的本质不同子串数。

令 k 为当前 s 的本质不同子串数。我们添加一个新的字符 c 至 s 的末尾。显然，会出现一些以 c 结尾的新的子串（以 c 结尾且之前未出现过的子串）。

设串 t 是 $s + c$ 的反串（反串指将原字符串的字符倒序排列形成的字符串）。我们的任务是计算有多少 t 的前缀未在 t 的其他地方出现。考虑计算 t 的 Z 函数并找到其最大值 $z_{\max}$ 。则 t 的长度小于等于 $z_{\max}$ 的前缀的反串在 s 中是已经出现过的以 c 结尾的子串。

所以，将字符 c 添加至 s 后新出现的子串数目为 $|t| - z_{\max}$ 。

算法时间复杂度为 $O(n^2)$ 。

值得注意的是，我们可以用同样的方法在 $O(n)$ 时间内，重新计算在端点处添加一个字符或者删除一个字符（从尾或者头）后的本质不同子串数目。

字符串整周期：

给定一个长度为 n 的字符串 s ，找到其最短的整周期，即寻找一个最短的字符串 t ，使得 s 可以被若干个 t 拼接而成的字符串表示。

考虑计算 s 的 Z 函数，则其整周期的长度为最小的 n 的因数 i ，满足 $i + z[i] = n$ 。

AC自动机

AC自动机： 时间复杂度 $O(\sum |s_i| + |s|)$

1. 用Trie树将所有模式串构成一颗trie
2. 用Kmp思想,构造失配指针来舍弃不匹配的前缀模式串,保留后缀模式串

3. *fail*每次跳都是跳到一个当前字符串的前缀,与当前后缀相同

```
1  struct AC { //默认小写字母, root 编号为1
2      struct node {
3          vector<int> son, endidx;
4          int fail, endcnt, in, cacnt, cnt;
5          node() {
6              son.resize(26);
7              fail = endcnt = in = cacnt = cnt = 0;
8          }
9      };
10
11     int n, idx = 1;
12     vector<node> tr;
13
14     AC() {
15         n = 0;
16         tr.resize(2);
17     }
18
19     void insert(string &s, int id = 0, int p = 1) {
20         for (int i = 0; i < s.size(); i++) {
21             int y = tr[p].son[s[i] - 'a'];
22             if (!y) {
23                 y = ++idx;
24                 tr.push_back(node());
25                 tr[p].son[s[i] - 'a'] = idx;
26             }
27             p = y;
28             tr[p].cnt++;
29         }
30         tr[p].endcnt++;
31         tr[p].endidx.push_back(id);
32         n++;
33     }
34
35     void getFail() {
36         for (int i = 0; i < 26; i++) tr[0].son[i] = 1;
37         queue<int> q;
38         q.push(1);
39         tr[1].fail = 0;
40
41         while (!q.empty()) {
42             int u = q.front();
43             q.pop();
44             for (int i = 0; i < 26; i++) {
45                 int v = tr[u].son[i];
46                 int F = tr[u].fail;
47                 if (!v) {
48                     tr[u].son[i] = tr[F].son[i];
49                     continue;
50                 }
51                 tr[v].fail = tr[F].son[i];
52                 tr[tr[v].fail].in++;
53                 q.push(v);
54             }
55         }
56     }
57
58     int query(string &s) { // 统计出现过多少种不同的模式串
59         int u = 1, ans = 0;
60         for (int i = 0; i < s.size(); i++) {
61             int v = s[i] - 'a';
62             int k = tr[u].son[v];
63             while (k > 1 && tr[k].endcnt != -1) {
64                 ans += tr[k].endcnt;
65                 tr[k].endcnt = -1;
```

```

66         k = tr[k].fail;
67     }
68     u = tr[u].son[v];
69 }
70 return ans;
71 }
72
73 vector<int> CountAppear(string &s) { // 统计每个模式串在字符串s中的出现次数, (模式串可能相同)
74     int u = 1;
75     vector<int> c(n + 1);
76     for (int i = 0; i < s.size(); i++) {
77         int v = s[i] - 'a';
78         int k = tr[u].son[v];
79         tr[k].cacnt++;
80         u = tr[u].son[v];
81     }
82
83     queue<int> q;
84     for (int i = 1; i <= idx; i++) {
85         if (!tr[i].in) q.push(i);
86     }
87
88     while (!q.empty()) { //拓扑优化
89         int u = q.front();
90         q.pop();
91         for (auto id : tr[u].endidx) {
92             c[id] += tr[u].cacnt;
93         }
94
95         int v = tr[u].fail;
96         tr[v].in--;
97         tr[v].cacnt += tr[u].cacnt;
98         if (!tr[v].in) q.push(v);
99     }
100
101     return c;
102 }
103
104 vector<int> CountAppearPerT() { // 统计每个模式串在别的模式串的出现次数
105     vector<int> c(n + 1);
106     queue<int> q;
107     for (int i = 1; i <= idx; i++) {
108         if (!tr[i].in) q.push(i);
109     }
110
111     while (!q.empty()) { //拓扑优化
112         int u = q.front();
113         q.pop();
114         for (auto id : tr[u].endidx) {
115             c[id] += tr[u].cnt;
116         }
117
118         int v = tr[u].fail;
119         tr[v].in--;
120         tr[v].cnt += tr[u].cnt;
121         if (!tr[v].in) q.push(v);
122     }
123     return c;
124 }
125 };

```

进行多模式匹配

```

1 struct trie_node {
2     array<int, 26> son = {}; //son是看字符
3     int exist = 0; //记录以该节点结尾的字符串个数
4     int fail = 0;

```

```

5   };
6
7   vector<trie_node> tr(1);
8
9   auto insert = [&](string& s) {
10       int p = 0;
11       for (auto x : s) {
12           int c = x - 'a';
13           if (!tr[p].son[c]) {
14               tr[p].son[c] = tr.size();    //若没有就添加节点
15               tr.push_back({});
16           }
17           p = tr[p].son[c];    //向下查
18       }
19       tr[p].exist++;    //记录此处为字符串
20   };
21
22   auto find = [&](string& s) {
23       int p = 0;
24       for (auto x : s) {
25           int c = x - 'a';
26           if (!tr[p].son[c]) {
27               return (int)0;
28           }
29           p = tr[p].son[c];
30       }
31       return tr[p].exist;
32   };
33
34   auto build = [&]() {    //建立AC自动机
35       queue<int> q;
36       for (int i = 0; i < 26; i++) {
37           if (tr[0].son[i]) {
38               q.push(tr[0].son[i]);
39           }
40       }
41       while (q.size()) {    //bfs建立AC自动机
42           int u = q.front();
43           q.pop();
44           for (int i = 0; i < 26; i++) {
45               if (tr[u].son[i]) {    //若存在tr[u][i],舍弃前缀直接匹配后缀模式串
46                   tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
47                   q.push(tr[u].son[i]);
48               } else {    //不存在直接连接,使得第一步操作的fail没有son[i]时可以找到下一个节点
49                   tr[u].son[i] = tr[tr[u].fail].son[i];
50               }
51           }
52       }
53   };
54
55   auto query = [&](string& s) {    //多模式匹配
56       int p = 0, res = 0;
57       for (auto x : s) {
58           int c = x - 'a';
59           p = tr[p].son[c];    //这里exist作用查看是否已经匹配过模式串,其他时候可删去
60           for (int j = p; j && tr[j].exist; j = tr[j].fail) {    //若匹配上则fail跳转
61               res += tr[j].exist;
62               tr[j].exist = 0;    //清空防止一个模式串匹配多次,有些情况不需要清空
63           }
64       }
65       return res;
66   };

```

拓扑排序优化建图： 预先记录,最后一并求和,避免每次都要跳 *fail*

```

1   int n;
2   cin >> n;

```

```

3
4 vector<string> a(n + 1);
5
6 struct trie_node {
7     array<int, 26> son;
8     int fail = 0, exist = 0, ans = 0;
9 };
10
11 int cnt = 0;
12 vector<trie_node> tr(2e5 + 10);
13 vector<int> rev(2e5 + 10), indeg(2e5 + 10), ans(2e5 + 10);
14
15 auto insert = [&](string & s, int num) {
16     int p = 0;
17     for (auto x : s) {
18         int c = x - 'a';
19         if (!tr[p].son[c]) {
20             tr[p].son[c] = ++cnt;
21         }
22         p = tr[p].son[c];
23     }
24     if (!tr[p].exist) {
25         tr[p].exist = num;
26     }
27     rev[num] = tr[p].exist;
28 };
29
30 auto getfail = [&]() {
31     queue<int> q;
32     for (int i = 0; i < 26; i++) {
33         if (tr[0].son[i]) {
34             q.push(tr[0].son[i]);
35         }
36     }
37     while (q.size()) {
38         int u = q.front();
39         q.pop();
40         int Fail = tr[u].fail;
41         for (int i = 0; i < 26; i++) {
42             int v = tr[u].son[i];
43             if (v) {
44                 tr[v].fail = tr[Fail].son[i];
45                 indeg[tr[Fail].son[i]]++;
46                 q.push(v);
47             } else {
48                 tr[u].son[i] = tr[Fail].son[i];
49             }
50         }
51     }
52 };
53
54 auto query = [&](string & s) {
55     int p = 0;
56     for (auto x : s) {
57         int c = x - 'a';
58         p = tr[p].son[c];
59         tr[p].ans++;
60     }
61 };
62
63 auto topu = [&]() {
64     queue<int> q;
65     for (int i = 0; i <= cnt; i++) {
66         if (!indeg[i]) {
67             q.push(i);
68         }
69     }

```

```

70     while (q.size()) {
71         int fr = q.front();
72         q.pop();
73         ans[tr[fr].exist] = tr[fr].ans;
74         int u = tr[fr].fail;
75         tr[u].ans += tr[fr].ans;
76         if (!(--indeg[u])) {
77             q.push(u);
78         }
79     }
80 };
81
82 for (int i = 1; i <= n; i++) {
83     cin >> a[i];
84     insert(a[i], i);
85 }
86
87 getfail();
88
89 string s;
90
91 cin >> s;
92
93 query(s);
94
95 topu();
96
97 for (int i = 1; i <= n; i++) {
98     cout << ans[rev[i]] << "\n";
99 }

```

后缀数组

后缀数组： 主要关系到两个数组： sa 和 rk 。

其中， $sa[i]$ 表示将所有后缀排序后第 i 小的后缀的编号，也是所说的后缀数组，后文也称编号数组 sa ；

$rk[i]$ 表示后缀 i 的排名，是重要的辅助数组，后文也称排名数组 rk 。

这两个数组满足性质： $sa[rk[i]] = rk[sa[i]] = i$ 。

height 数组：

LCP (最长公共前缀)：

两个字符串 S 和 T 的 LCP 就是最大的 x ($x \leq \min(|S|, |T|)$) 使得 $S_i = T_i$ ($\forall 1 \leq i \leq x$)。

下文中以 $\text{lcp}(i, j)$ 表示后缀 i 和后缀 j 的最长公共前缀 (的长度)。

height 数组的定义：

$\text{height}[i] = \text{lcp}(sa[i], sa[i - 1])$ ，即第 i 名的后缀与它前一名后缀的最长公共前缀。

$\text{height}[1]$ 可以视作 0。

```

1  struct SuffixArray {
2      int n;
3      std::vector<int> sa, rk, lc;
4      SuffixArray(const std::string &s) {
5          n = s.length();
6          sa.resize(n);
7          lc.resize(n - 1);
8          rk.resize(n);
9          std::iota(sa.begin(), sa.end(), 0);
10         std::sort(sa.begin(), sa.end(), [&](int a, int b) {return s[a] < s[b];});
11         rk[sa[0]] = 0;
12         for (int i = 1; i < n; ++i)
13             rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);

```

```

14     int k = 1;
15     std::vector<int> tmp, cnt(n);
16     tmp.reserve(n);
17     while (rk[sa[n - 1]] < n - 1) {
18         tmp.clear();
19         for (int i = 0; i < k; ++i)
20             tmp.push_back(n - k + i);
21         for (auto i : sa)
22             if (i >= k)
23                 tmp.push_back(i - k);
24         std::fill(cnt.begin(), cnt.end(), 0);
25         for (int i = 0; i < n; ++i)
26             ++cnt[rk[i]];
27         for (int i = 1; i < n; ++i)
28             cnt[i] += cnt[i - 1];
29         for (int i = n - 1; i >= 0; --i)
30             sa[--cnt[rk[tmp[i]]]] = tmp[i];
31         std::swap(rk, tmp);
32         rk[sa[0]] = 0;
33         for (int i = 1; i < n; ++i)
34             rk[sa[i]] = rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i - 1] + k == n ||
tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
35         k *= 2;
36     }
37     for (int i = 0, j = 0; i < n; ++i) {
38         if (rk[i] == 0) {
39             j = 0;
40         } else {
41             for (j -= j > 0; i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] +
j]; )
42                 ++j;
43             lc[rk[i] - 1] = j;
44         }
45     }
46 }
47 };

```

寻找最小的循环移动位置：

将字符串 S 复制一份变成 SS 就转化成了后缀排序问题。

例题：「JSOI2007」字符加密。

在字符串中找子串：

任务是在线地在主串 T 中寻找模式串 S 。在线的意思是，我们已经预先知道主串 T ，但是当且仅当询问时才知道模式串 S 。我们可以先构造出 T 的后缀数组，然后查找子串 S 。若子串 S 在 T 中出现，它必定是 T 的一些后缀的前缀。因为我们已经将所有后缀排序了，我们可以通过在 p 数组中二分 S 来实现。比较子串 S 和当前后缀的时间复杂度为 $O(|S|)$ ，因此找子串的时间复杂度为 $O(|S| \log |T|)$ 。注意，如果该子串在 T 中出现了多次，每次出现都是在 p 数组中相邻的。因此出现次数可以通过再次二分找到，输出每次出现的位置也很轻松。

从字符串首尾取字符最小化字典序：

暴力做法就是每次最坏 $O(n)$ 地判断当前应该取首还是尾（即比较取首得到的字符串与取尾得到的反串的大小），只需优化这一判断过程即可。

由于需要在原串后缀与反串后缀构成的集合内比较大小，可以将反串拼接在原串后，并在中间加上一个没出现过的字符（如 $\#$ ，代码中可以直接使用空字符），求后缀数组，即可 $O(1)$ 完成这一判断。

两子串最长公共前缀：

$$\text{lcp}(\text{sa}[i], \text{sa}[j]) = \min\{\text{height}[i + 1..j]\}$$

感性理解：如果 height 一直大于某个数，前这么多位就一直没变过；反之，由于后缀已经排好序了，不可能变了之后变回来。

严格证明可以参考[2004] 后缀数组 by 徐智磊。

有了这个定理，求两子串最长公共前缀就转化为了 RMQ 问题。

比较一个字符串的两个子串的大小关系：

假设需要比较的是 $A = S[a..b]$ 和 $B = S[c..d]$ 的大小关系。

若 $\text{lcp}(a, c) \geq \min(|A|, |B|)$, 则 $A < B \iff |A| < |B|$ 。

否则, $A < B \iff \text{rk}[a] < \text{rk}[c]$ 。

不同子串的数目:

子串就是后缀的前缀, 所以可以枚举每个后缀, 计算前缀总数, 再减掉重复。

「前缀总数」其实就是子串个数, 为 $\frac{n(n+1)}{2}$ 。

如果按后缀排序的顺序枚举后缀, 每次新增的子串就是除了与上一个后缀的 LCP 剩下的前缀。这些前缀一定是新增的, 否则会破坏 $\text{lcp}(\text{sa}[i], \text{sa}[j]) = \min\{\text{height}[i+1..j]\}$ 的性质。只有这些前缀是新增的, 因为 LCP 部分在枚举上一个前缀时计算过了。

所以答案为:

$$\frac{n(n+1)}{2} - \sum_{i=2}^n \text{height}[i]$$

出现至少 k 次的子串的最大长度:

出现至少 k 次意味着后缀排序后有至少连续 k 个后缀以这个子串作为公共前缀。

所以, 求出每相邻 $k-1$ 个 height 的最小值, 再求这些最小值的最大值就是答案。

可以使用单调队列 $O(n)$ 解决, 但使用其它方式也足以 AC。

是否有某字符串在文本串中至少不重叠地出现了两次:

可以二分目标串的长度 $|s|$, 将 h 数组划分成若干个连续 LCP 大于等于 $|s|$ 的段, 利用 RMQ 对每个段求其中出现的数中最大和最小的下标, 若这两个下标的距离满足条件, 则一定有长度为 $|s|$ 的字符串不重叠地出现了两次。

连续的若干个相同子串:

我们可以枚举连续串的长度 $|s|$, 按照 $|s|$ 对整个串进行分块, 对相邻两块的块首进行 LCP 与 LCS 查询, 具体可见[2009] 后缀数组——处理字符串的有力工具。

结合并查集:

某些题目求解时要求你将后缀数组划分成若干个连续 LCP 长度大于等于某一值的段, 亦即将 h 数组划分成若干个连续最小值大于等于某一值的段并统计每一段的答案。如果有多次询问, 我们可以将询问离线。观察到当给定值单调递减的时候, 满足条件的区间个数总是越来越少, 而新区间都是两个或多个原区间相连所得, 且新区间中不包含在原区间内的部分的 h 值都为减少到的这个值。我们只需要维护一个并查集, 每次合并相邻的两个区间, 并维护统计信息即可。

结合线段树:

某些题目让你求满足条件的前若干个, 而这些数又在后缀排序中的一个区间内。这时我们可以用归并排序的性质来合并两个结点的信息, 利用线段树维护和查询区间答案。

结合单调栈:

被加数的前两项很好处理, 为 $\frac{n(n-1)(n+1)}{2}$ (每个后缀都出现了 $n-1$ 次, 后缀总长是 $\frac{n(n+1)}{2}$), 关键是最后一项, 即后缀的两两 LCP。

我们知道 $\text{lcp}(i, j) = k$ 等价于 $\min\{\text{height}[i+1..j]\} = k$ 。所以, 可以把 $\text{lcp}(i, j)$ 记作 $\min\{x \mid i+1 \leq x \leq j, \text{height}[x] = \text{lcp}(i, j)\}$ 对答案的贡献。

考虑每个位置对答案的贡献是哪些后缀的 LCP, 其实就是从它开始向左若干个连续的 height 大于它的后缀中选一个, 再从向右若干个连续的 height 不小于它的后缀中选一个。这个东西可以用单调栈计算。

后缀自动机

一些前置约定/定义:

记 Σ 为字符集, $|\Sigma|$ 为字符集大小。对于一个字符串 s , 记 $|s|$ 为其长度。

后缀自动机概述:

后缀自动机 (suffix automaton, SAM) 是一个能解决许多字符串相关问题的有力的数据结构。

举个例子, 以下的字符串问题都可以在线性时间内通过 SAM 解决:

1. 在另一个字符串中搜索一个字符串的所有出现位置。
2. 计算给定的字符串中有多少个不同的子串。

直观上，字符串的 SAM 可以理解成给定字符串的所有子串的压缩形式。值得注意的事实是，SAM 将所有的这些信息以高度压缩的形式储存。对于一个长度为 n 的字符串，它的空间复杂度仅为 $O(n)$ 。此外，构造 SAM 的时间复杂度仅为 $O(n)$ 。准确地说，一个 SAM 最多有 $2n - 1$ 个节点和 $3n - 4$ 条转移边。

定义：

字符串 s 的 SAM 是一个接受 s 的所有后缀的最小 DFA（确定性有限自动机或确定性有限状态自动机）。

换句话说：

- SAM 是一张有向无环图。结点被称作状态，边被称作状态间的转移。
- 图存在一个源点 t_0 ，称作初始状态，其它各结点均可从 t_0 出发到达。
- 每个转移都标有一些字母。从一个结点出发的所有转移均不同。
- 存在一个或多个终止状态。如果从初始状态 t_0 出发，最终转移到了一个终止状态，则路径上的所有转移连接起来一定是字符串 s 的一个后缀。 s 的每个后缀均可用一条从 t_0 到某个终止状态的路径构成。
- 在所有满足上述条件的自动机中，SAM 的结点数是最少的。

子串的性质：

SAM 最简单、也最重要的性质是，它包含关于字符串 s 的所有子串的信息。任意从初始状态 t_0 开始的路径，如果我们将转移路径上的标号写下来，都会形成 s 的一个子串。反之每个 s 的子串对应从 t_0 开始的某条路径。

为了简化表达，我们称子串对应一条路径（从 t_0 开始、由一些标号构成这个子串）。反过来，我们说任意一条路径对应它的标号构成的字符串。

到达某个状态的路径可能不止一条，因此我们说一个状态对应一些字符串的集合，这个集合的每个元素对应这些路径。

后缀链接 link：

考虑 SAM 中某个不是 t_0 的状态 v 。我们已经知道，状态 v 对应于具有相同 endpos 的等价类。我们如果定义 w 为这些字符串中最长的一个，则所有其它的字符串都是 w 的后缀。

我们还知道字符串 w 的前几个后缀（按长度降序考虑）全部包含于这个等价类，且所有其它后缀（至少有一个——空后缀）在其它的等价类中。我们记 t 为最长的这样的后缀，然后将 v 的后缀链接连到 t 上。

换句话说，一个后缀链接 $\text{link}(v)$ 连接到对应于 w 的最长后缀的另一个 endpos 等价类的状态。

状态数：

对于一个长度为 n 的字符串 s ，它的 SAM 中的状态数不会超过 $2n - 1$ （假设 $n \geq 2$ ）。

算法本身即可证明该结论。一开始，自动机含有一个状态，第一次和第二次迭代中只会创建一个节点，剩余的 $n - 2$ 步中每步会创建至多 2 个状态。

然而我们也能在不借助这个算法的情况下证明这个估计值。我们回忆一下状态数等于不同的 endpos 集合个数。这些 endpos 集合形成了一棵树（祖先节点的集合包含了它所有孩子节点的集合）。考虑将这棵树稍微变形一下：只要它有一个只有一个孩子的内部结点（这意味着孩子节点的集合至少遗漏了它的父集合中的一个位置），我们创建一个含有这个遗漏位置的集合。最后我们可以获得一棵每一个内部结点的度数大于 1 的树，且叶子节点的个数不超过 n 。因此这样的树里有不超过 $2n - 1$ 个节点。

字符串 $\text{abbb} \cdots \text{bbb}$ 的状态数达到了该上界：从第三次迭代后的每次迭代，算法都会拆开一个状态，最终产生恰好 $2n - 1$ 个状态。

转移数：

对于一个长度为 n 的字符串 s ，它的 SAM 中的转移数不会超过 $3n - 4$ （假设 $n \geq 3$ ）。

额外信息：

观察实现中的结构体的每个变量。实际上，尽管 SAM 本身由 next 组成，但 SAM 构造算法中作为辅助变量的 link 和 len 在应用中常常比 next 重要，甚至可以抛开 next 单独使用。

设字符串的长度为 n ，考虑 extend 操作中 cur 变量的值，这个节点对应的状态是执行 extend 操作时的当前字符串，即字符串的一个前缀，每个前缀有一个终点。这样得到的 n 个节点，对应了 n 个不同的终点。设第 i 个节点为 v_i ，对应的是 $S_{1..i}$ ，终点是 i 。姑且把这些节点称之为「终点节点」。

考虑给 SAM 赋予树形结构，树的根为 0，且其余节点 v 的父亲为 $\text{link}(v)$ 。则这棵树与原 SAM 的关系是：

- 每个节点的终点集合等于其子树内所有终点节点对应的终点的集合。

- 在此基础上可以给每个节点赋予一个最长字符串，是其终点集合中任意一个终点开始往前取 len 个字符得到的字符串。每个这样的字符串都一样，且 len 恰好是满足这个条件的最大值。

这些字符串满足的性质是：

- 如果节点 A 是 B 的祖先，则节点 A 对应的字符串是节点 B 对应的字符串的后缀。
- 这条性质把字符串所有前缀组成了一棵树，且有许多符合直觉的树的性质。例如， $S_{1..p}$ 和 $S_{1..q}$ 的最长公共后缀对应的字符串就是 v_p 和 v_q 对应的 LCA 的字符串。实际上，这棵树与将字符串 S 翻转后得到字符串的压缩后缀树结构相同。

每个状态 i 对应的子串数量是 $\text{len}(i) - \text{len}(\text{link}(i))$ （节点 0 例外）。注意到 $\text{link}(i)$ 对应的字符串是 i 对应的字符串的一个后缀，这些子串就是 i 对应字符串的所有后缀，去掉被父亲「抢掉」的那部分，即 $\text{link}(i)$ 对应字符串的所有后缀。

```

1  struct SAM {
2      static constexpr int ALPHABET_SIZE = 26;
3      struct Node {
4          int len;
5          int link;
6          std::array<int, ALPHABET_SIZE> next;
7          Node() : len{}, link{}, next{} {}
8      };
9      std::vector<Node> t;
10     SAM() {
11         init();
12     }
13     void init() {
14         t.assign(2, Node());
15         t[0].next.fill(1);
16         t[0].len = -1;
17     }
18     int newNode() {
19         t.emplace_back();
20         return t.size() - 1;
21     }
22     int extend(int p, int c) {
23         if (t[p].next[c]) {
24             int q = t[p].next[c];
25             if (t[q].len == t[p].len + 1) {
26                 return q;
27             }
28             int r = newNode();
29             t[r].len = t[p].len + 1;
30             t[r].link = t[q].link;
31             t[r].next = t[q].next;
32             t[q].link = r;
33             while (t[p].next[c] == q) {
34                 t[p].next[c] = r;
35                 p = t[p].link;
36             }
37             return r;
38         }
39         int cur = newNode();
40         t[cur].len = t[p].len + 1;
41         while (!t[p].next[c]) {
42             t[p].next[c] = cur;
43             p = t[p].link;
44         }
45         t[cur].link = extend(p, c);
46         return cur;
47     }
48     int extend(int p, char c, char offset = 'a') {
49         return extend(p, c - offset);
50     }
51
52     int next(int p, int x) {
53         return t[p].next[x];
54     }
55 }

```

```

56     int next(int p, char c, char offset = 'a') {
57         return next(p, c - 'a');
58     }
59
60     int link(int p) {
61         return t[p].link;
62     }
63
64     int len(int p) {
65         return t[p].len;
66     }
67
68     int size() {
69         return t.size();
70     }
71 };

```

应用：

下面我们来看一些可以用 SAM 解决的问题。简单起见，假设字符集的大小 k 为常数。这允许我们认为增加一个字符和遍历的复杂度为常数。

检查字符串是否出现：

给一个文本串 T 和多个模式串 P ，我们要检查字符串 P 是否作为 T 的一个子串出现。

我们在 $O(|T|)$ 的时间内对文本串 T 构造后缀自动机。为了检查模式串 P 是否在 T 中出现，我们沿转移（边）从 t_0 开始根据 P 的字符进行转移。如果在某个点无法转移下去，则模式串 P 不是 T 的一个子串。如果我们能够这样处理完整个字符串 P ，那么模式串在 T 中出现过。

对于每个字符串 P ，算法的时间复杂度为 $O(|P|)$ 。此外，这个算法还找到了模式串 P 在文本串中出现的最大前缀长度。

不同子串个数：

给一个字符串 S ，计算不同子串的个数。

对字符串 S 构造后缀自动机。

每个 S 的子串都相当于自动机中的一些路径。因此不同子串的个数等于自动机中以 t_0 为起点的不同路径的条数。

考虑到 SAM 为有向无环图，不同路径的条数可以通过动态规划计算。即令 d_v 为从状态 v 开始的路径数量（包括长度为零的路径），则我们有如下递推方程：

$$d_v = 1 + \sum_{w: (v,w,c) \in \text{DAWG}} d_w$$

即， d_v 可以表示为所有 v 的转移的末端的和。

所以不同子串的个数为 $d_{t_0} - 1$ （因为要去掉空子串）。

总时间复杂度为： $O(|S|)$ 。

另一种方法是利用上述后缀自动机的树形结构。每个节点对应的子串数量是 $\text{len}(i) - \text{len}(\text{link}(i))$ ，对自动机所有节点求和即可

所有不同子串的总长度：

给定一个字符串 S ，计算所有不同子串的总长度。

本题做法与上一题类似，只是现在我们需要考虑分两部分进行动态规划：不同子串的数量 d_v 和它们的总长度 ans_v 。

我们已经在上一题中介绍了如何计算 d_v 。 ans_v 的值可以通过以下递推式计算：

$$ans_v = \sum_{w: (v,w,c) \in \text{DAWG}} d_w + ans_w$$

我们取每个邻接结点 w 的答案，并加上 d_w （因为从状态 v 出发的子串都增加了一个字符）。

算法的时间复杂度仍然是 $O(|S|)$ 。

同样可以利用上述后缀自动机的树形结构。每个节点对应的所有后缀长度是

$$\frac{\text{len}(i) \times (\text{len}(i) + 1)}{2}$$

减去其 link 节点的对值就是该节点的净贡献，对自动机所有节点求和即可。

字典序第 k 大子串:

给定一个字符串 S 。多组询问，每组询问给定一个数 K_i ，查询 S 的所有子串中字典序第 K_i 大的子串。

这个问题的思路可以从解决前两个问题的思路发展而来。字典序第 k 大的子串对应于 SAM 中字典序第 k 大的路径，因此在计算每个状态的路径数后，我们可以很容易地从 SAM 的根开始找到第 k 大的路径。

预处理的时间复杂度为 $O(|S|)$ ，单次查询的复杂度为 $O(|ans| \cdot |\Sigma|)$ （其中 ans 是查询的答案， $|\Sigma|$ 为字符集的大小）。

虽然该题是后缀自动机的经典题，但实际上这题由于涉及字典序，用后缀数组做最方便。

例题：SPOJ - SUBLEX，TJOI2015 弦论

最小循环移位:

给定一个字符串 S 。找出字典序最小的循环移位。

容易发现字符串 $S + S$ 包含字符串 S 的所有循环移位作为子串。

所以问题简化为在 $S + S$ 对应的后缀自动机上寻找最小的长度为 $|S|$ 的路径，这可以通过平凡的方法做到：我们从初始状态开始，贪心地访问最小的字符即可。

总的时间复杂度为 $O(|S|)$ 。

出现次数:

对于一个给定的文本串 T ，有多组询问，每组询问给一个模式串 P ，回答模式串 P 在字符串 T 中作为子串出现了多少次。

利用后缀自动机的树形结构，进行 dfs 即可预处理每个节点的终点集合大小。在自动机上查找模式串 P 对应的节点，如果存在，则答案就是该节点的终点集合大小；如果不存在，则答案为 0。

以下为原方法：

对文本串 T 构造后缀自动机。

接下来做预处理：对于自动机中的每个状态 v ，预处理 cnt_v ，使之等于 $\text{endpos}(v)$ 集合的大小。事实上，对应同一状态 v 的所有子串在文本串 T 中的出现次数相同，这相当于集合 endpos 中的位置数。

然而我们不能明确的构造集合 endpos ，因此我们只考虑它们的大小 cnt 。

为了计算这些值，我们进行以下操作。对于每个状态，如果它不是通过复制创建的（且它不是初始状态 t_0 ），我们将它的 cnt 初始化为 1。然后我们按它们的长度 len 降序遍历所有状态，并将当前的 cnt_v 的值加到后缀链接指向的状态上，即：

$$cnt_{\text{link}(v)} += cnt_v$$

这样做每个状态的答案都是正确的。

为什么这是正确的？不是通过复制获得的状态，恰好有 $|T|$ 个，并且它们中的前 i 个在我们插入前 i 个字符时产生。因此对于每个这样的状态，我们在它被处理时计算它所对应的位置的数量。因此我们初始将这些状态的 cnt 的值赋为 1，其它状态的 cnt 值赋为 0。

接下来我们对每一个 v 执行以下操作： $cnt_{\text{link}(v)} += cnt_v$ 。其背后的含义是，如果有一个字符串 v 出现了 cnt_v 次，那么它的所有后缀也在完全相同的地方结束，即也出现了 cnt_v 次。

为什么我们在这个过程中不会重复计数（即把某些位置数了两次）呢？因为我们只将一个状态的位置添加到一个其它的状态上，所以一个状态不可能以两种不同的方式将其位置重复地指向另一个状态。

因此，我们可以在 $O(|T|)$ 的时间内计算出所有状态的 cnt 的值。

最后回答询问只需要查找值 cnt_t ，其中 t 为模式串对应的状态，如果该模式串不存在答案就为 0。单次查询的时间复杂度为 $O(|P|)$ 。

第一次出现的位置:

给定一个文本串 T ，多组查询。每次查询字符串 P 在字符串 T 中第一次出现的位置（ P 的开头位置）。

我们构造一个后缀自动机。我们对 SAM 中的所有状态预处理位置 firstpos 。即，对每个状态 v 我们想要找到第一次出现这个状态的末端的位置 $\text{firstpos}[v]$ 。换句话说，我们希望先找到每个集合 endpos 中的最小的元素（显然我们不能显式地维护所有 endpos 集合）。

为了维护 firstpos 这些位置，我们将原函数扩展为 `Error: ' ' allowed only in math mode` 当我们创建新状态 cur 时，我们令：

$$\text{firstpos}(cur) = \text{len}(cur) - 1$$

当我们将结点 q 复制到 $clone$ 时，我们令：

$$\text{firstpos}(clone) = \text{firstpos}(q)$$

(因为值的唯一的其它选项 $\text{firstpos}(cur)$ 显然太大了)。

那么查询的答案就是 $\text{firstpos}(t) - |P| + 1$ ，其中 t 为对应字符串 P 的状态。单次查询只需要 $O(|P|)$ 的时间。

所有出现的位置：

问题同上，这一次需要查询文本串 T 中模式串出现的所有位置。

利用后缀自动机的树形结构，遍历子树，一旦发现终点节点就输出。

以下为原解法：

我们还是对文本串 T 构造后缀自动机。与上一个问题相似，我们为所有状态计算位置 firstpos 。

如果 t 为对应于模式串 P 的状态，显然 $\text{firstpos}(t)$ 为答案的一部分。需要查找的其它位置怎么办？我们使用了含有字符串 P 的自动机，我们还需要将哪些状态纳入自动机呢？所有对应于以 P 为后缀的字符串的状态。换句话说我们要找到所有可以通过后缀链接到达状态 t 的状态。

因此为了解决这个问题，我们需要为每一个状态保存一个指向它的后缀引用列表。查询的答案就包含了对于每个我们能从状态 t 只使用后缀引用进行 DFS 或 BFS 的所有状态的 firstpos 值。

这种变通方案的时间复杂度为 $O(\text{answer}(P))$ ，因为我们不会重复访问一个状态（因为对于仅有一个后缀链接指向一个状态，所以不存在两条不同的路径指向同一状态）。

我们只需要考虑两个可能有相同 endpos 值的不同状态。如果一个状态是由另一个复制而来的，则这种情况会发生。然而，这并不会对复杂度分析造成影响，因为每个状态至多被复制一次。

此外，如果我们不从被复制的节点输出位置，我们也可以去除重复的位置。事实上对于一个状态，如果经过被复制状态可以到达，则经过原状态也可以到达。因此，如果我们给每个状态记录标记 `is_clone` 来代表这个状态是不是被复制出来的，我们就可以简单地忽略掉被复制的状态，只输出其它所有状态的 firstpos 的值。

```
1  struct state {
2      bool is_clone;
3      int first_pos;
4      std::vector<int> inv_link;
5      // some other variables
6  };
7
8  // 在构造 SAM 后
9  for (int v = 1; v < sz; v++) st[st[v].link].inv_link.push_back(v);
10
11 // 输出所有出现位置
12 void output_all_occurrences(int v, int P_length) {
13     if (!st[v].is_clone) cout << st[v].first_pos - P_length + 1 << endl;
14     for (int u : st[v].inv_link) output_all_occurrences(u, P_length);
15 }
```

最短的没有出现的字符串：

给定一个字符串 S 和一个特定的字符集，我们要找一个长度最短的没有在 S 中出现过的字符串。

我们在字符串 S 的后缀自动机上做动态规划。

令 d_v 为节点 v 的答案，即，我们已经处理完了子串的一部分，当前在状态 v ，想找到不连续的转移需要添加的最小字符数量。计算 d_v 非常简单。如果不存在使用字符集中至少一个字符的转移，则 $d_v = 1$ 。否则添加一个字符是不够的，我们需要求出所有转移中的最小值：

$$d_v = 1 + \min_{w:(v,w,c) \in \text{SAM}} d_w$$

问题的答案就是 d_{t_0} ，字符串可以通过计算过的数组 d 逆推回去。

两个字符串的最长公共子串：

给定两个字符串 S 和 T ，求出最长公共子串，公共子串定义为在 S 和 T 中都作为子串出现过的字符串 X 。

我们对字符串 S 构造后缀自动机。

我们现在处理字符串 T ，对于每一个前缀，都在 S 中寻找这个前缀的最长后缀。换句话说，对于每个字符串 T 中的位置，我们要找到这个位置结束的 S 和 T 的最长公共子串的长度。显然问题的答案就是所有 l 的最大值。

为了达到这一目的，我们使用两个变量，当前状态 v 和当前长度 l 。这两个变量描述当前匹配的部分：它的长度和它们对应的状态。

一开始 $v = t_0$ 且 $l = 0$ ，即，匹配为空串。

现在我们来描述如何添加一个字符 T_i 并为其重新计算答案：

1. 如果存在一个从 v 到字符 T_i 的转移，我们只需要转移并让 l 自增一。
2. 如果不存在这样的转移，我们需要缩短当前匹配的部分，这意味着我们需要按照后缀链接进行转移：

$$v = \text{link}(v)$$

与此同时，需要缩短当前长度。显然我们需要将 l 赋值为 $\text{len}(v)$ ，因为经过这个后缀链接后我们到达的状态所对应的最长字符串是一个子串。

如果仍然没有使用这一字符的转移，我们继续重复经过后缀链接并减小 l ，直到我们找到一个转移或到达虚拟状态 -1 （这意味着字符 T_i 根本没有在 S 中出现过，所以我们设置 $v = l = 0$ ）。

这一部分的时间复杂度为 $O(|T|)$ ，因为每次移动我们要么可以使 l 增加一，要么可以在后缀链接间移动几次，每次都减小 l 的值。

```
1  string lcs(const string &S, const string &T) {
2      sam_init();
3      for (int i = 0; i < S.size(); i++) sam_extend(S[i]);
4
5      int v = 0, l = 0, best = 0, bestpos = 0;
6      for (int i = 0; i < T.size(); i++) {
7          while (v && !st[v].next.count(T[i])) {
8              v = st[v].link;
9              l = st[v].length;
10         }
11         if (st[v].next.count(T[i])) {
12             v = st[v].next[T[i]];
13             l++;
14         }
15         if (l > best) {
16             best = l;
17             bestpos = i;
18         }
19     }
20     return T.substr(bestpos - best + 1, best);
21 }
```

多个字符串间的最长公共子串：

给定 k 个字符串 S_i 。我们需要找到它们的最长公共子串，即作为子串出现在每个字符串中的字符串 X 。

我们将所有的子串连接成一个较长的字符串 T ，以特殊字符 D_i 分开每个字符串（一个字符对应一个字符串）：

$$T = S_1 + D_1 + S_2 + D_2 + \dots + S_k + D_k$$

然后对字符串 T 构造后缀自动机。

现在我们需要在自动机中找到存在于所有字符串 S_i 中的一个字符串，这可以通过使用添加的特殊字符完成。注意如果 S_j 包含了一个子串，则 SAM 中存在一条从包含字符 D_j 的子串而不包含以其它字符 $D_1, \dots, D_{j-1}, D_{j+1}, \dots, D_k$ 开始的路径。

因此我们需要计算可达性，即对于自动机中的每个状态和每个字符 D_i ，是否存在这样的一条路径。这可以容易地通过 DFS 或 BFS 及动态规划计算。之后，问题的答案就是状态 v 的字符串 $\text{longest}(v)$ 中存在所有特殊字符的路径。

回文自动机

定义：

和其它自动机类似的，回文树也是由转移边和后缀链接（fail 指针）组成，每个节点都可以代表一个回文子串。

因为回文串长度分为奇数和偶数，我们可以像 manacher 那样加入一个不在字符集中的字符（如 '#'）作为分隔符来将所有回文串的长度都变为奇数，但是这样过于麻烦了。有没有更好的办法呢？

答案自然是有。更好的办法就是建两棵树，一棵树中的节点对应的回文子串长度均为奇数，另一棵树中的节点对应的回文子串长度均为偶数。

和其它的自动机一样，一个节点的 fail 指针指向的是这个节点所代表的回文串的最长回文后缀所对应的节点，但是转移边并非代表在原节点代表的回文串后加一个字符，而是表示在原节点代表的回文串前后各加一个相同的字符（不难理解，因为要保证存的是回文串）。

我们还需要在每个节点上维护此节点对应回文子串的长度 len，这个信息保证了我们可以轻松地构造出回文树。

```
1  struct PAM {
2      static constexpr int ALPHABET_SIZE = 28;
3      struct Node {
4          int len;
5          int link;
6          int cnt;
7          std::array<int, ALPHABET_SIZE> next;
8          Node() : len{}, link{}, cnt{}, next{} {}
9      };
10     std::vector<Node> t;
11     int suff;
12     std::string s;
13     PAM() {
14         init();
15     }
16     void init() {
17         t.assign(2, Node());
18         t[0].len = -1;
19         suff = 1;
20         s.clear();
21     }
22     int newNode() {
23         t.emplace_back();
24         return t.size() - 1;
25     }
26
27     bool add(char c, char offset = 'a') {
28         int pos = s.size();
29         s += c;
30         int let = c - offset;
31         int cur = suff, curlen = 0;
32
33         while (true) {
34             curlen = t[cur].len;
35             if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos])
36                 break;
37             cur = t[cur].link;
38         }
39         if (t[cur].next[let]) {
40             suff = t[cur].next[let];
41             return false;
42         }
43
44         int num = newNode();
45         suff = num;
46         t[num].len = t[cur].len + 2;
47         t[cur].next[let] = num;
48
49         if (t[num].len == 1) {
50             t[num].link = 1;
51             t[num].cnt = 1;
52             return true;
53         }
54
55         while (true) {
56             cur = t[cur].link;
```

```

57         curlen = t[cur].len;
58         if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
59             t[num].link = t[cur].next[let];
60             break;
61         }
62     }
63
64     t[num].cnt = 1 + t[t[num].link].cnt;
65
66     return true;
67 }
68 };
69
70 PAM pam;

```

```

1  string s;
2  cin >> s;
3
4  s = "$" + s;
5
6  struct trie_node {
7      array<int, 26> son = {};
8      int len = 0, fail = 0, exist = 0;
9  };
10
11  int tot = 1, last = 0;
12  vector<trie_node> pam;
13
14  auto new_node = [&](int x) {
15      int now = ++tot;
16      pam[now] = {{}, x, 0, 0};
17      return now;
18  };
19
20  auto getfail = [&](string & s, int x, int n) {
21      while (s[n - pam[x].len - 1] != s[n]) {
22          x = pam[x].fail;
23      }
24      return x;
25  };
26
27  auto insert = [&](int c, int pos) {
28      int cur = getfail(s, last, pos);
29
30      if (!pam[cur].son[c]) {
31          int now = new_node(pam[cur].len + 2);
32          pam[now].fail = pam[getfail(s, pam[cur].fail, pos)].son[c];
33          pam[cur].son[c] = now;
34      }
35
36      last = pam[cur].son[c];
37
38      pam[last].exist++;
39  };
40
41  auto build = [&](string & s) {
42      pam = vector<trie_node>(s.size() + 2);
43      tot = 1, last = 0;
44
45      pam[0] = {{}, 0, 1, 0}, pam[1] = {{}, -1, 0, 0};
46
47      for (int i = 1; i < s.size(); i++) {
48          insert(s[i] - 'a', i);
49      }
50  };
51
52  build(s);

```

```

53
54  int ans = 0;
55
56  for (int i = tot; i >= 0; i--) {
57      ans = max(ans, pam[i].exist * pam[i].len);
58      pam[pam[i].fail].exist += pam[i].exist;
59  }
60
61  cout << ans << "\n";

```

本质不同回文子串个数:

由线性状态数的证明，容易知道一个串的本质不同回文子串个数等于回文树的状态数（排除奇根和偶根两个状态）。

回文子串出现次数:

建出回文树，使用类似后缀自动机统计出现次数的方法。

由于回文树的构造过程中，节点本身就是按照拓扑序插入，因此只需要逆序枚举所有状态，将当前状态的出现次数加到其 fail 指针对应状态的出现次数上即可。

例题：「APIO2014」回文串

定义:

s 的一个子串的存在值为这个子串在 s 中出现的次数乘以这个子串的长度。对于给定的字符串 s ，求所有回文子串中的最大存在值。

公式:

令 s 的回文树为 pam ，节点 i 的长度为 $\text{len}(i)$ ，出现次数为 $\text{exist}(i)$ ，失配指针为 $\text{fail}(i)$ 。则最大存在值为：

$$\max_i (\text{exist}(i) \times \text{len}(i))$$

最小回文划分:

给定一个字符串 s ($1 \leq |s| \leq 10^5$)，求最小的 k ，使得存在 $s_1, s_2, \dots, s_k$ ，满足 s_i ($1 \leq i \leq k$) 均为回文串，且 $s_1, s_2, \dots, s_k$ 依次连接后得到的字符串等于 s 。

考虑动态规划，记 $\text{dp}[i]$ 表示 s 长度为 i 的前缀的最小划分数，转移只需要枚举以第 i 个字符结尾的所有回文串。

公式:

$$\text{dp}[i] = 1 + \min_{s[j+1..i] \text{ 为回文串}} \text{dp}[j]$$

由于一个字符串最多会有 $O(n^2)$ 个回文子串，因此上述算法的时间复杂度为 $O(n^2)$ ，无法接受。为了优化转移过程，下面给出一些引理。

记字符串 s 长度为 i 的前缀为 $\text{pre}(s, i)$ ，长度为 i 的后缀为 $\text{suf}(s, i)$ 。

定义:

- **周期**：若 $0 < p \leq |s|$ ， $\forall 1 \leq i \leq |s| - p$ ， $s[i] = s[i + p]$ ，就称 p 是 s 的周期。
- **border**：若 $0 \leq r < |s|$ ， $\text{pre}(s, r) = \text{suf}(s, r)$ ，就称 $\text{pre}(s, r)$ 是 s 的 border。

周期和 border 的关系:

t 是 s 的 border，当且仅当 $|s| - |t|$ 是 s 的周期。

推论:

s 的所有回文后缀按照长度排序后，可以划分成 $\log |s|$ 段等差数列。

优化:

回文树上的每个节点 u 需要多维护两个信息， $\text{diff}[u]$ 和 $\text{slink}[u]$ 。 $\text{diff}[u]$ 表示节点 u 和 $\text{fail}[u]$ 所代表的回文串的长度差，即 $\text{len}[u] - \text{len}[\text{fail}[u]]$ 。 $\text{slink}[u]$ 表示 u 一直沿着 fail 向上跳到第一个节点 v ，使得 $\text{diff}[v] \neq \text{diff}[u]$ ，也就是 u 所在等差数列中长度最小的那个节点。

根据上面证明的结论，如果使用 slink 指针向上跳的话，每向后添加一个字符，只需要向上跳 $O(\log |s|)$ 次。因此，可以考虑将一个等差数列表示的所有回文串的 dp 值之和（在原问题中指 $\min$ ），记录到最长的那一个回文串对应节点上。

$g[v]$ 表示 v 所在等差数列的 dp 值之和，且 v 是这个等差数列中长度最长的节点，则

$$g[v] = \sum_{\text{slink}[x]=\text{slink}[v]} \text{dp}[i - \text{len}[x]]$$

这里 i 是当前枚举到的下标。

下面我们考虑如何更新 g 数组和 dp 数组。以下图为例，假设当前枚举到第 i 个字符，回文树上对应节点为 x 。 $g[x]$ 为橙色三个位置的 dp 值之和（最短的回文串 $\text{slink}[x]$ 算在下一个等差数列中）。 $\text{fail}[x]$ 上一次出现位置是 $i - \text{diff}[x]$ （在 $i - \text{diff}[x]$ 处结束）， $g[\text{fail}[x]]$ 包含的 dp 值是蓝色位置。因此， $g[x]$ 实际上等于 $g[\text{fail}[x]]$ 和多出来一个位置的 dp 值之和，多出来的位置是 $i - (\text{len}[\text{slink}[x]] + \text{diff}[x])$ 。最后再用 $g[x]$ 去更新 $\text{dp}[i]$ ，这部分等差数列的贡献就计算完毕了，不断跳 $\text{slink}[x]$ ，重复这个过程即可。具体实现方式可参考例题代码。

最后，上述做法的正确性依赖于：如果 x 和 $\text{fail}[x]$ 属于同一个等差数列，那么 $\text{fail}[x]$ 上一次出现位置是 $i - \text{diff}[x]$ 。